

Online Shopping System

Objectives

- Design an Online Shopping System using OOP.
- Demonstrate classes and objects for Products, Customers and Orders.
- Apply encapsulation, inheritance, polymorphism and abstraction.
- Explain constructors, class methods, interfaces and abstract classes.
- Evaluate the benefits of OOP for maintainability and scalable software.

Coursework Presentation

Aasiyah Rahman | Student ID: 12701771 | June 2026



Defining Classes and Objects

- A **class** is a blueprint for objects that define attributes and methods of an object (Sridhar, Indumathi and Hariharan, 2023).
- An **object** is an instance of a class. Each object has a state (attributes) and behaviour (methods) (Sridhar, Indumathi and Hariharan, 2023).

```
product1 = Product("Bunny Teddy", 29.99, 20)  
product2 = Product("Bear Teddy", 34.99, 25)
```

product1 and product2 are objects created from the Product class. Each object stores its own name, price and stock values.

Figure 1: UML Class Diagram



Figure 2: Product Display Output

Products available:

Product Name: Bunny Teddy
Price: £29.99
Stock: 20

Product Name: Bear Teddy
Price: £34.99
Stock: 25

Product Name: Elephant Teddy
Price: £32.99
Stock: 15

Total products in system: 3



Constructors and Destructors

- **Constructors** (`__init__`) initialise object attributes when an object is created (Python Software Foundation, 2026).
- **Destructors** (`__del__`) execute when an object is removed from memory (Python Software Foundation, 2026).
- These methods ensure objects being with valid data and can perform cleanup operations, managing an objects lifecycle ([Sridhar](#), [Indumathi](#) and [Hariharan](#), 2023).

The Product constructor ensures every product starts with valid attributes before being added to the system.

The destructor displays a message when products or orders are removed from memory.

Figure 3: Constructor for Product Class

```
def __init__(self, name, price, stock):  
    self.__name = name  
    self.__price = price  
    self.stock = stock  
    Product.product_count += 1
```

Figure 4: Destructor for Product Class

```
def __del__(self):  
    print(f"\nProduct '{self.name}' has been removed from the system")
```

Figure 5: Destructor Output When Program Terminates

```
Product 'Bunny Teddy' has been removed from the system  
Product 'Bear Teddy' has been removed from the system  
Product 'Elephant Teddy' has been removed from the system  
Order 101 has been removed from the system.
```



Class Methods and Instance Variables

- **Instance variables** are values of variables that differ in each object.
- **Class variables** are shared between all instances of a class.

(Sridhar, Indumathi and Hariharan, 2023)

The Product class uses the shared `product_count` class variable to track the total number of products created. The Order class uses instance methods such as `add_product()` and `calculate_total()` to manage order-specific data and calculations.

Figure 6: Class Variable and Class Method

```
product_count = 0
@classmethod
def total_products(cls):
    print(f"\nTotal products in system: {cls.product_count}")
```

Figure 7: Output of Class Method

```
Total products in system: 3
```

Figure 8: Instance Method for Order Calculations

```
def calculate_total(self):
    total = 0
    for product in self.products:
        total += product.price
    return total
```

Figure 9: Premium Customer Order Calculation

```
Subtotal: £64.98
Premium Customer Discount (10%): -£6.50
Current order total: £58.48
```



Understanding Inheritance Relationships

- **Inheritance** allows a child class to reuse attributes and methods from a parent class (Sridhar, Indumathi and Hariharan, 2023).

The inheritance hierarchy in this online shopping system uses multilevel inheritance: User → Customer → PremiumCustomer.

Figure 10: Inheritance Hierarchy

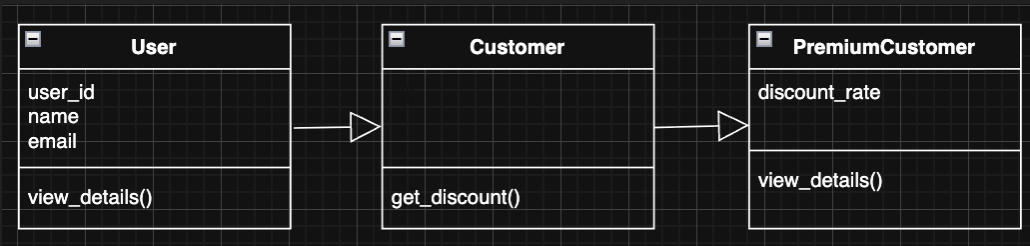


Figure 11: PremiumCustomer Inheritance

```
class PremiumCustomer(Customer):
    def __init__(self, user_id, name, email, discount_rate):
        super().__init__(user_id, name, email)
        self.__discount_rate = discount_rate
```

Figure 12: PremiumCustomer Output

User Information:

```
Premium User ID: 1
Premium User Name: aasiyah
Premium User Email: aasiyah@gmail.com
Premium User Discount Rate: 10%
```



Method Overriding and Polymorphic Behaviour

- **Method Overriding** occurs when a child class provides its own implementation of a method inherited from a parent class.
- **Polymorphism** allows the same method name to behave differently depending on the object type.

Sridhar, Indumathi and Hariharan, 2023

Customer and PremiumCustomer both implement view_details().

get_discount_rate() also behaves differently for standard and premium customers.

Figure 13: Customer Version

```
def view_details(self):
    print("\nUser Information: ")
    print(f"\nUser ID: {self.user_id}")
    print(f"User Name: {self.name}")
    print(f"User Email: {self.email}")

def get_discount_rate(self):
    return 0
```

Figure 15: PremiumCustomer Override

```
def view_details(self):
    print("\nUser Information: ")
    print(f"\nPremium User ID: {self.user_id}")
    print(f"Premium User Name: {self.name}")
    print(f"Premium User Email: {self.email}")
    print(f"Premium User Discount Rate: {self.discount_rate}%")

def get_discount_rate(self):
    return self.__discount_rate
```

Figure 14: Standard Customer Output

```
User Information:

User ID: 3
User Name: aasiyah
User Email: aasiyah@gmail.com
```

Figure 16: PremiumCustomer Output

```
User Information:

Premium User ID: 4
Premium User Name: aasiyah
Premium User Email: aasiyah@gmail.com
Premium User Discount Rate: 10%
```



Define an abstract class User and an interface Orderable

- **Abstract classes** define common behaviour for subclasses and cannot be instantiated directly because they contain incomplete implementations (Lott, 2024).

User cannot be instantiated directly because it contains an abstract method.

Orderable acts as a interface requiring;

- place_order()
- calculate_total()

Classes inheriting from these abstractions must utilise the required methods.

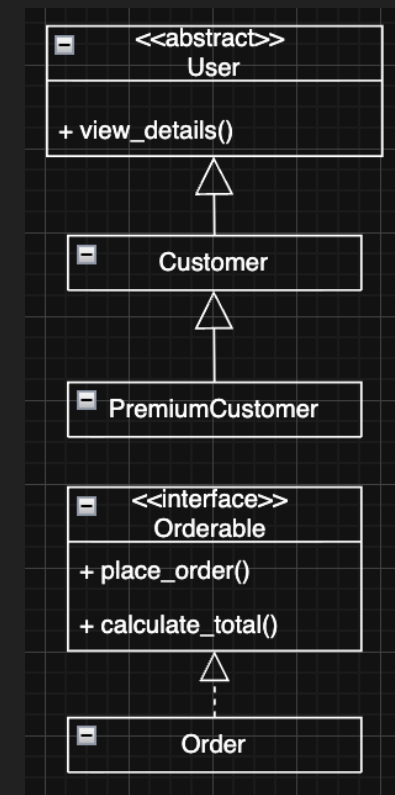
Figure 17: Abstract User Class

```
class User(ABC):  
  
    @abstractmethod  
    def view_details(self):  
        pass
```

Figure 18: Orderable Interface

```
class Orderable(ABC):  
  
    @abstractmethod  
    def place_order(self):  
        pass  
  
    @abstractmethod  
    def calculate_total(self):  
        pass
```

Figure 19: Abstract Class and Interface UML Diagram



Access Modifiers and Data Hiding

- **Access modifiers** control how class attributes and methods can be accessed.
- Python supports private (`__`), protected (`_`) and public attributes.
- Private attributes use the prefix `__` to restrict direct access to data (Sridhar, Indumathi and Hariharan, 2023).
- These properties provide controlled access to private attributes.
- **Data hiding** restricts direct access to internal class data through the use of access modifiers (Sridhar, Indumathi and Hariharan, 2023).

In this system, Order uses private attributes such as `__order_id`, `__products` and `__status` to protect order data from direct modification.

Figure 20: Access Modifier Comparison

Access Type	Purpose	Example
Private	Hidden from direct access	<code>__status</code>
Protected	Internal use by class/subclass	<code>_stock</code>
Public	Accessible everywhere	<code>product_count</code>

Figure 21: Private Order Attributes

```
self.__order_id = order_id
self.__products = []
self.__customer = None
self.__status = "Pending"
```

Figure 22: Property Method for Controlled Access

```
@property
def status(self):
    return self.__status
```



Encapsulation and Information Hiding

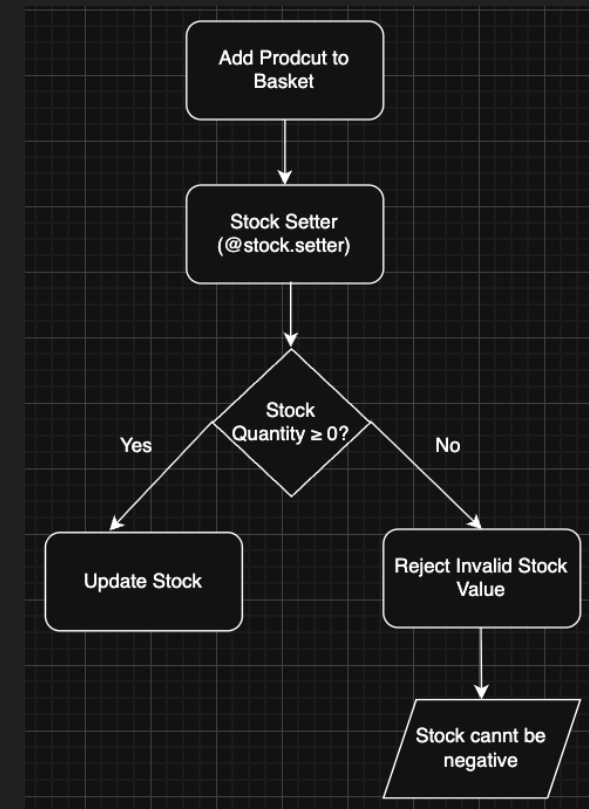
- **Encapsulation** combines data and methods within a class and restricts direct access to internal information through data hiding (Sridhar, Indumathi and Hariharan, 2023).

The Product class controls stock updates through properties and setters. Validation prevents invalid stock values from entering the system. Stock can only be modified through controlled methods rather than direct access.

Figure 23: Product Stock Setter Method

```
@stock.setter
def stock(self, quantity):
    if quantity >= 0:
        self._stock = quantity
    else:
        print("\nStock cannot be negative.")
```

Figure 24: Product Stock Validation Process



Abstract Data Type and Interfaces

- An **abstract data type** specifies available operations while hiding implementation details.
- Abstraction allows users to focus on required operations without needing to understand internal implementation details (Sridhar, Indumathi and Hariharan, 2023).

Users interact with the Order class through methods such as:

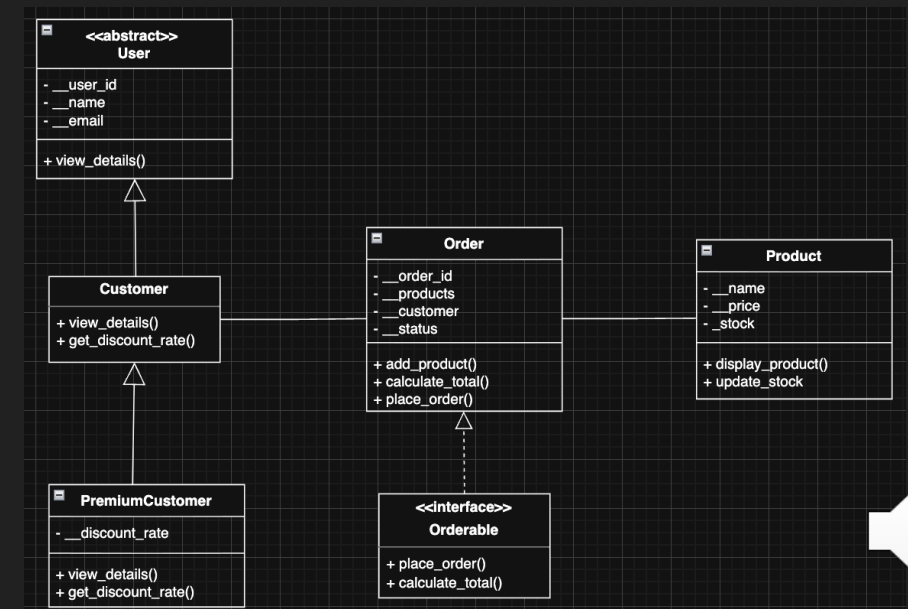
- add_product()
- calculate_total()
- place_order()

Orderable acts as an interface defining required behavior.

Figure 25: Orderable Interface with Abstract Methods

```
class Orderable(ABC):  
  
    @abstractmethod  
    def place_order(self):  
        pass  
  
    @abstractmethod  
    def calculate_total(self):  
        pass
```

Figure 26: Full UML Class Diagram of the Online Shopping System



Conclusion

Summary

- Classes and objects model products, customers and orders.
- Inheritance and polymorphism reduce code duplication and improve flexibility.
- Encapsulation protects data integrity through controlled access.
- Abstraction and interfaces improve maintainability and scalability.

Strength of Design

- Abstract classes and interfaces enforce consistent behavior and support future extensibility.
- Encapsulation and validation protect data integrity by preventing invalid values.
- Inheritance reduces code duplication through code reuse.
- Polymorphism improves flexibility and simplifies future expansion.

Limitations of Design

- Only one active order can exist per session.
- Data is lost when program closes
- No persistent storage for customers, products or orders.
- Limited to command-line interface

Future Improvements

- Database integration.
- Multiple orders per customer.
- Payment processing.
- Additional order types implementing Orderable.



Q&A

Questions?

References

- Lott, S.F. (2024) *Modern Python Cookbook: 130+ Updated Recipes for Modern Python 3.12 with New Techniques and Tools*. 3rd edn. Birmingham: Packt Publishing Ltd. Available at: <https://learning.oreilly.com/library/view/modern-python-cookbook/9781835466384/> (Accessed: 16 June 2026).
- Python Software Foundation (2026) *Data model*. Available at: <https://docs.python.org/3/reference/datamodel.html> (Accessed: 10 June 2026).
- Sridhar, S., Indumathi, J. and Hariharan, V.M. (2023) *Python Programming*. Noida: Pearson India. Available at: <https://learning.oreilly.com/library/view/python-programming-by/9789357053303/> (Accessed: 6 June 2026).